

LINK AL REPOSITORIO EN GITHUB:

<https://github.com/pabloke1987-hash/TPI-Sistema-de-Gestion-de-Paises>

LINK AL VIDEO DEMOSTRATIVO:

<https://youtu.be/BntPv5HF-mg>

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Programación I

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

ÍNDICE ORIENTATIVO

1. Introducción y objetivo del programa	_____	pag. 2
2. Marco teórico	_____	pag. 2
3. Diseño del caso práctico	_____	pag. 3
4. Metodología utilizada	_____	pag. 8
5. Resultados obtenidos	_____	pag. 9
6. Conclusiones	_____	pag. 11
7. Repositorio en GitHub	_____	pag. 11
8. ANEXO: Explicación desarrollada de funciones clave	_____	pag. 12
9. Bibliografía	_____	pag. 15

1. Introducción y objetivo del programa

El programa desarrollado es un sistema de gestión de datos de países implementado en Python. Su finalidad es administrar un conjunto de registros almacenados en un archivo CSV, permitiendo al usuario consultar, agregar, actualizar, eliminar, filtrar, ordenar y analizar información mediante estadísticas básicas. Para contextualizar el caso práctico se utilizó un dataset de 48 países asociados a la Copa Mundial de Fútbol 2026. Esta decisión permite trabajar con un volumen de datos suficiente para demostrar búsquedas, filtros por continente, rangos numéricos, ordenamientos y estadísticas comparativas. El dataset utilizado está compuesto por países representados mediante cuatro datos: nombre, población, superficie y continente. Esta decisión respeta la consigna del trabajo, ya que el archivo CSV no incorpora columnas adicionales ni información que no haya sido solicitada. Desde el punto de vista académico, el programa permite demostrar el uso integrado de varios contenidos de Programación I: listas, diccionarios, funciones, estructuras condicionales, ciclos, archivos, validaciones, manejo de excepciones, ordenamientos y cálculos estadísticos.

2. Marco teórico

El marco teórico presenta los conceptos de programación utilizados para desarrollar el sistema. No se trata solamente de una lista de definiciones, sino de la relación entre los contenidos vistos en Programación I y su aplicación concreta dentro del proyecto.

Programación modular y uso de funciones

La programación modular consiste en dividir un programa en partes más pequeñas, organizadas y reutilizables. En Python, esta organización se logra mediante funciones, que agrupan instrucciones relacionadas con una tarea específica. En este proyecto se aplicó este criterio para separar la carga de datos, la interfaz, las validaciones, las operaciones CRUD, los filtros, los ordenamientos, las estadísticas y la persistencia. Esta decisión mejora la legibilidad del código, facilita las pruebas y evita repetir instrucciones. La función `main()` actúa como coordinadora general del sistema, mientras que cada función secundaria resuelve una responsabilidad puntual.

Listas y diccionarios en Python

Las listas permiten almacenar colecciones de elementos y recorrerlos mediante ciclos. Los diccionarios, por su parte, permiten representar información mediante pares clave-valor. La documentación oficial de Python presenta estas estructuras como herramientas fundamentales para organizar datos, recorrer colecciones y construir nuevas listas mediante comprensiones [1]. En el proyecto se combinan ambas estructuras: la lista contiene todos los países y cada país se representa mediante un diccionario con las claves nombre, población, superficie y continente. Esta estructura resulta adecuada porque mantiene agrupados los atributos de cada país y permite acceder a ellos por nombre de campo.

Ejemplo de estructura utilizada: `pais = {"nombre": "Argentina", "poblacion": 47000000, "superficie": 2780400, "continente": "America"}`

Archivos CSV y persistencia de datos

La persistencia de datos permite que la información continúe disponible después de cerrar el programa. En este trabajo se utiliza un archivo CSV, formato de texto tabular que puede ser leído y editado por distintas herramientas. Python incluye el módulo csv, que permite leer y escribir archivos CSV, incluyendo el trabajo con diccionarios mediante DictReader y DictWriter [2]. El uso de CSV resulta apropiado para este nivel del proyecto porque no requiere instalar una base de datos y permite visualizar fácilmente los registros en programas como Excel o editores de texto.

Validación de datos y manejo de errores

La validación consiste en comprobar que los datos ingresados o leídos sean correctos antes de usarlos o guardarlos. En el sistema se validan campos obligatorios, valores numéricos, duplicados, encabezados del CSV, registros dañados y selecciones fuera de rango. El manejo de errores mediante try/except permite controlar situaciones inesperadas sin que el programa finalice abruptamente. La documentación de Python explica que las excepciones pueden ser capturadas por cláusulas except, permitiendo responder de forma controlada ante errores de ejecución [3].

Ordenamientos, búsquedas y estadísticas

El programa utiliza búsquedas secuenciales para encontrar países por nombre, filtros para seleccionar subconjuntos de datos, sorted() para ordenar registros y funciones integradas como max(), min(), sum() y len() para obtener indicadores estadísticos. Estas operaciones permiten transformar la lista de países en información útil para el usuario.

Buenas prácticas de programación

La guía PEP 8 propone convenciones de estilo para escribir código Python de forma legible y consistente [4]. En este proyecto se aplican buenas prácticas como nombres descriptivos, separación en funciones, uso de constantes, comentarios claros, validaciones y mensajes comprensibles para el usuario.

3. Diseño del Caso Práctico

Análisis de la consigna

El diseño del caso práctico partió del análisis del enunciado y de la identificación de las funcionalidades obligatorias. A partir de esa lectura se definió que el sistema debía administrar información de países, permitir operaciones sobre los registros y conservar los cambios realizados por el usuario. Para evitar una solución desordenada, se planificó una arquitectura modular. El programa no se desarrolló como un único bloque de instrucciones, sino como un conjunto de funciones especializadas que interactúan entre sí a través de parámetros, valores de retorno y modificaciones controladas sobre la lista principal de países.

Dataset utilizado

Como base de datos inicial se utilizó un conjunto de 48 países. Esta cantidad permite demostrar adecuadamente el funcionamiento de los filtros, ordenamientos y estadísticas. Cada registro contiene cuatro atributos: nombre, población, superficie y continente.

Campo	Descripción	Ejemplo
nombre	Nombre del país registrado.	Argentina
población	Cantidad de habitantes representada como número entero.	47000000
superficie	Superficie del país en km ² , representada como número entero.	2780400
continente	Continente al que pertenece el país.	America

Estructura de datos principal: lista de diccionarios

La estructura central del programa es una lista de diccionarios. La lista funciona como una colección que contiene todos los países cargados en memoria. Cada elemento de esa lista es un diccionario que representa un país individual. Esta estructura es adecuada porque cada país posee varios atributos relacionados entre sí. Si se utilizaran listas separadas para nombres, poblaciones, superficies y continentes, el programa sería más propenso a errores. En cambio, el diccionario agrupa todos los datos de un país en una sola unidad lógica.

Ventaja de la estructura elegida

La lista permite recorrer todos los registros con `for`, agregar países con `append()`, eliminar registros con `remove()`, aplicar filtros mediante comprensiones de listas y ordenar la información con `sorted()`.

Arquitectura general del programa

La arquitectura del sistema está organizada de forma modular. Esto significa que el programa no está escrito como un único bloque largo de instrucciones, sino que se divide en funciones especializadas. Cada función cumple una tarea concreta y se relaciona con las demás a través de parámetros, valores de retorno o modificaciones sobre la lista principal de países. El punto de entrada del programa es la función `main()`. Desde allí se cargan los datos, se muestra el menú principal y se derivan las operaciones según la opción elegida por el usuario. Esta estructura centralizada facilita entender el flujo del sistema. Esta organización permite separar claramente la carga de datos, la interfaz, las validaciones, la persistencia en CSV y las operaciones funcionales del sistema.

Módulo	Funciones principales	Responsabilidad
Configuración	ARCHIVO_CSV, CAMPOS, ANCHO_INTERFAZ, PAISES_INICIALES	Definir constantes y datos iniciales.
Interfaz	mostrar_linea, mostrar_titulo, mostrar_lista_paises, mostrar_menu_principal	Mostrar información de forma clara.
Formato y normalización	normalizar_texto, capitalizar_texto, formatear_numero	Preparar textos y números para comparar o mostrar.
Validación	leer_texto_obligatorio, leer_entero, validar_registro	Controlar datos ingresados o leídos.
Persistencia	crear_csv_inicial, cargar_paises, guardar_paises	Leer y escribir el archivo CSV.
Búsqueda	existe_pais, buscar_paises_por_nombre, seleccionar_pais_de_resultados	Encontrar registros.
CRUD	agregar_pais, buscar_pais, actualizar_pais, eliminar_pais	Gestionar altas, consultas, modificaciones y bajas.
Filtros	filtrar_por_continente, filtrar_por_rango, menu_filtros	Mostrar subconjuntos de datos.
Ordenamientos	ordenar_paises, menu_ordenamientos	Ordenar registros por distintos criterios.

Estadísticas	calcular_cantidad_por_continente, mostrar_estadisticas	Generar indicadores del dataset.
Control general	main	Coordinar todo el flujo del programa.

Nota Importante

El desarrollo del funcionamiento de la diferentes funciones se encuentra agregado en el apartado ANEXO, al final del informe.

Flujo completo de ejecución del programa

El funcionamiento del sistema comienza cuando Python ejecuta el bloque principal del programa mediante la instrucción `if __name__ == "__main__":`, desde donde se invoca la función `main()`. Esta función actúa como punto de control de toda la aplicación y coordina el resto de las operaciones. En primer lugar, el sistema ejecuta `cargar_paises(ARCHIVO_CSV)`, encargada de verificar la existencia del archivo `paises.csv`, leer su contenido y cargar los registros en una lista de diccionarios que permanecerá en memoria durante la ejecución. Si el archivo no existe, el sistema crea automáticamente un nuevo archivo con la estructura necesaria para comenzar a trabajar. Luego se informa al usuario cuántos países fueron cargados y se presenta el menú principal dentro de un ciclo `while True`. Este ciclo permite realizar múltiples operaciones sin reiniciar el programa. Cada opción seleccionada invoca una función específica. Cuando una operación modifica los datos, se actualiza la lista en memoria y se llama a `guardar_paises()` para persistir los cambios en el CSV. Al seleccionar la opción 0 - Salir, el programa guarda los datos, muestra un mensaje de despedida y finaliza correctamente.

Paso	Acción principal
1	Inicio del programa y ejecución de <code>main()</code> .
2	Verificación y carga del archivo <code>paises.csv</code> .
3	Creación de la lista de países en memoria.
4	Presentación del menú principal.
5	Selección de una opción por parte del usuario.

6	Ejecución de la función correspondiente.
7	Validación y procesamiento de la información.
8	Actualización de la lista en memoria cuando corresponde.
9	Persistencia de los cambios mediante guardar_paises().
10	Regreso al menú principal o finalización del programa.

Persistencia mediante archivos CSV

La persistencia de datos significa que la información no se pierde cuando el programa termina. En este proyecto, la persistencia se logra mediante el archivo paises.csv. Este archivo contiene una fila de encabezados y una fila por cada país. Cuando el programa inicia, cargar_paises() abre el archivo CSV y transforma cada fila en un diccionario de Python. Cuando el usuario agrega, actualiza o elimina un país, guardar_paises() vuelve a escribir el archivo para conservar los cambios.

Decisión técnica

Se eligió CSV porque es simple, liviano, portable y adecuado para un trabajo introductorio de Programación I. Además, permite comprobar visualmente los cambios realizados por el programa.

Validaciones y manejo de errores

Una parte fundamental del programa es evitar que el usuario ingrese datos incorrectos. Las validaciones protegen la integridad del archivo CSV y de la lista de países en memoria. Sin estas validaciones, el sistema podría guardar registros incompletos, números inválidos o países duplicados.

Validación	Función donde aparece	Motivo
Campo vacío	leer_texto_obligatorio	Evita nombres o continentes sin contenido.
Texto solo numérico	leer_texto_obligatorio	Evita que el nombre de un país sea un número.
Número no entero	leer_entero	Evita errores al convertir con int().

Valor menor al mínimo	leer_entero	Impide población negativa o superficie inválida.
País duplicado	existe_pais / agregar_pais	Evita repetir registros.
CSV con encabezados incorrectos	validar_encabezados_csv	Garantiza que el archivo respete la estructura esperada.
Registro del CSV inválido	validar_registro	Evita cargar filas dañadas.
Selección fuera de rango	seleccionar_pais_de_resultados	Evita elegir una posición inexistente.

Diseño de la interfaz de consola

La interfaz fue mejorada para que el usuario pueda comprender fácilmente qué está haciendo el sistema. Se utilizan separadores visuales, títulos en mayúsculas, emojis informativos y tablas alineadas. Estos elementos no cambian la lógica del programa, pero mejoran la experiencia de uso y dan una presentación más profesional al trabajo. La función `mostrar_lista_paises()` es especialmente importante porque centraliza la forma en que se presentan los datos. Al definir anchos fijos para cada columna, la tabla se mantiene alineada aunque existan nombres largos como República Democrática del Congo o Bosnia y Herzegovina.

Documentación gráfica complementaria

Los diagramas de flujo, BPMN 2.0, arquitectura del programa y diagramas de secuencia no se incorporan dentro del informe principal para evitar sobrecargar el documento. Estos materiales se anexan en el repositorio GitHub como documentación técnica complementaria.

4. Metodología utilizada

La metodología de trabajo fue incremental. Primero se identificaron los requisitos de la consigna y luego se construyó el programa por etapas, probando cada funcionalidad antes de avanzar con la siguiente. Este enfoque permitió detectar errores de validación, mejorar la interfaz y mantener una organización clara del código.

Etapas	Actividad realizada	Resultado
1	Análisis de la consigna.	Identificación de funcionalidades obligatorias y restricciones.

2	Diseño de la estructura de datos.	Definición de una lista de diccionarios como estructura central.
3	Construcción del CSV inicial.	Creación del archivo paises.csv con registros base.
4	Desarrollo de funciones.	Implementación de carga, guardado, búsqueda, filtros, ordenamientos y estadísticas.
5	Validación de entradas.	Control de campos vacíos, duplicados y valores numéricos inválidos.
6	Pruebas funcionales.	Ejecución de casos con datos válidos e inválidos.
7	Mejora de interfaz.	Ajuste de mensajes, tablas y presentación visual en consola.
8	Documentación y repositorio.	Preparación del README, informe, diagramas y archivos complementarios.

Criterios de prueba aplicados

- Probar el inicio del programa con el archivo CSV existente y con el archivo ausente.
- Verificar que el listado muestre correctamente todos los registros cargados.
- Intentar agregar países válidos y países duplicados.
- Probar entradas incorrectas como letras en campos numéricos o campos vacíos.
- Actualizar países existentes y comprobar que los cambios se guarden en el CSV.
- Eliminar países previa confirmación del usuario.
- Ejecutar filtros por continente, población y superficie.
- Ejecutar ordenamientos por nombre, población y superficie.
- Consultar estadísticas y verificar que se calculen sobre todos los registros cargados.

5. Resultados obtenidos

El resultado final es un sistema funcional de consola que permite administrar registros de países almacenados en un archivo CSV. El programa cumple con las operaciones requeridas, conserva los cambios entre ejecuciones y presenta mensajes de validación claros para el usuario. La muestra de las pruebas de funcionalidad están desarrolladas en el video explicativo que viene anexo con la entrega de este informe.

Funcionalidad	Resultado alcanzado
Listar países	Muestra todos los registros en una tabla alineada.
Agregar país	Permite incorporar nuevos registros validando campos obligatorios y duplicados.
Actualizar país	Permite modificar población y superficie de un país existente.
Buscar país	Permite encontrar coincidencias por nombre parcial o completo.
Filtrar países	Permite filtrar por continente, rango de población y rango de superficie.
Ordenar países	Permite ordenar por nombre, población o superficie.
Estadísticas	Calcula mayor y menor población, promedios y cantidad por continente.
Eliminar país	Permite eliminar un registro solicitando confirmación.
Persistencia CSV	Los cambios se guardan en paises.csv y se recuperan al reiniciar.
Validaciones	Se controlan campos vacíos, números inválidos, duplicados y errores del CSV.

Nota Importante

La demostración del funcionamiento del programa está desarrollada en el video explicativo y demostrativo que se adjunta con la entrega del informe.

6. Conclusiones

El desarrollo del Sistema de Gestión de Países permitió integrar contenidos fundamentales de Programación I en un caso práctico completo. A través del proyecto se aplicaron estructuras de datos, funciones, ciclos, condicionales, manejo de archivos, validaciones, excepciones, búsquedas, filtros, ordenamientos y cálculos estadísticos. Una de las decisiones más importantes fue utilizar una lista de diccionarios como estructura principal, ya que permite representar cada país de forma clara y manejar sus atributos de manera ordenada. Otra decisión relevante fue utilizar un archivo CSV para lograr persistencia de datos, evitando que la información se pierda al finalizar el programa. El trabajo también permitió comprender la importancia de la modularización. Al separar el código en funciones, el programa resulta más legible, más fácil de probar y más sencillo de modificar. Además, las validaciones implementadas mejoran la calidad del sistema porque evitan datos incompletos, duplicados o inválidos. Como mejora futura, el sistema podría ampliarse con una interfaz gráfica, una base de datos relacional, exportación de reportes o filtros más avanzados. Sin embargo, para el alcance de la consigna, el programa cumple los objetivos planteados y demuestra el uso integrado de los principales contenidos de la materia.

7. Repositorio en GitHub

El repositorio en GitHub forma parte de la entrega del trabajo práctico y tiene el código fuente completo, el archivo CSV, el README y la documentación complementaria del proyecto.

Elemento	Descripción
main.py	Archivo principal del programa. Contiene la lógica del sistema y las funciones implementadas.
países.csv	Archivo de datos utilizado para la persistencia.
README.md	Documento de presentación del proyecto, instrucciones de ejecución y explicación de funcionalidades.
/Diagramas	Carpeta sugerida para incluir diagramas de flujo, BPMN 2.0, arquitectura y secuencia.
/Documentación	Carpeta sugerida para informe y material complementario.

LINKS DE ACCESO

Link del enlace al Repositorio en GITHUB:

<https://github.com/pabloke1987-hash/TPI-Sistema-de-Gestion-de-Paises>

Link del enlace al video explicativo en YouTube:

<https://youtu.be/BntPv5HF-mg>

8. ANEXO: Explicación desarrollada de funciones clave

- **cargar_paises(ruta_archivo)**

Objetivo: Carga la información desde el archivo CSV y la transforma en una lista de diccionarios.

Proceso interno: Primero crea una lista vacía llamada paises. Luego verifica si el archivo existe. Si no existe, llama a crear_csv_inicial() para reconstruirlo con PAISES_INICIALES. Después abre el archivo, usa csv.DictReader para leerlo y valida que los encabezados sean correctos. Cada fila se pasa por validar_registro(). Si el registro es válido, se agrega a la lista.

Interacción con otras funciones: Interactúa con crear_csv_inicial(), validar_encabezados_csv() y validar_registro(). Devuelve la lista que será usada por todo el programa.

Conceptos aplicados: CSV, listas, diccionarios, ciclos for, condicionales, return, manejo de errores.

- **guardar_paises(ruta_archivo, paises)**

Objetivo: Guarda en el CSV la lista actualizada de países.

Proceso interno: Abre el archivo en modo escritura, crea un csv.DictWriter con los campos definidos en CAMPOS, escribe el encabezado y luego escribe todos los diccionarios de la lista.

Interacción con otras funciones: Es llamada después de agregar, modificar o eliminar registros. Garantiza que los cambios no se pierdan al cerrar el programa.

Conceptos aplicados: Archivos, DictWriter, listas de diccionarios, persistencia de datos.

- **mostrar_lista_paises(paises)**

Objetivo: Muestra todos los países en una tabla alineada.

Proceso interno: Primero verifica si la lista está vacía. Si tiene datos, define anchos fijos para cada columna, construye una línea de tabla, imprime encabezados en mayúsculas y recorre cada país con enumerate(). Los números se formatean con formatear_numero().

Interacción con otras funciones: Se utiliza en listados generales, búsquedas, filtros y ordenamientos. Mejora la experiencia visual del usuario.

Conceptos aplicados: Ciclos for, enumerate, f-strings, formateo de texto, listas, diccionarios.

- **agregar_pais(paises)**

Objetivo: Permite incorporar un nuevo país al sistema.

Proceso interno: Solicita nombre, población, superficie y continente. Valida que el nombre no esté vacío, que los números sean correctos y que el país no exista previamente. Si todo es válido, crea un diccionario y lo agrega a la lista con append(). Luego llama a guardar_paises().

Interacción con otras funciones: Usa leer_texto_obligatorio(), leer_entero(), existe_pais(), capitalizar_texto() y guardar_paises().

Conceptos aplicados: CRUD Create, validaciones, append, diccionarios, persistencia.

- **buscar_pais(paises)**

Objetivo: Permite consultar países por nombre o por una parte del nombre.

Proceso interno: Solicita un término de búsqueda y llama a buscar_paises_por_nombre(). Si encuentra resultados, los muestra con mostrar_lista_paises(). Si no encuentra nada, informa al usuario.

Interacción con otras funciones: Reutiliza la búsqueda parcial y la tabla de salida.

Conceptos aplicados: Búsqueda secuencial, condicionales, listas por comprensión.

- **actualizar_pais(paises)**

Objetivo: Actualiza la población y superficie de un país existente.

Proceso interno: Busca coincidencias por nombre. Si hay una sola, la selecciona directamente. Si hay varias, permite elegir una. Luego muestra el registro actual, solicita nuevos valores y guarda los cambios.

Interacción con otras funciones: Interactúa con buscar_paises_por_nombre(), seleccionar_pais_de_resultados(), mostrar_pais(), leer_entero() y guardar_paises().

Conceptos aplicados: CRUD Update, búsqueda, validación numérica, modificación de diccionarios.

- **eliminar_pais(paises)**

Objetivo: Elimina un país de la lista y del archivo CSV.

Proceso interno: Busca el país, permite seleccionar entre varias coincidencias y solicita confirmación. Si el usuario confirma, elimina el diccionario de la lista con remove() y guarda el CSV actualizado.

Interacción con otras funciones: Interactúa con funciones de búsqueda, visualización y persistencia.

Conceptos aplicados: CRUD Delete, remove, condicionales, confirmación de usuario.

- **filtrar_por_rango(paises, campo, titulo)**

Objetivo: Filtra países según un rango numérico de población o superficie.

Proceso interno: Solicita un valor mínimo y uno máximo. Valida que el mínimo no sea mayor que el máximo. Luego recorre la lista y selecciona los países cuyo valor en el campo indicado esté dentro del rango.

Interacción con otras funciones: Es reutilizada por el menú de filtros para población y superficie, evitando duplicar código.

Conceptos aplicados: Parámetros, listas por comprensión, condicionales, reutilización de funciones.

- **ordenar_paises(paises, campo, descendente=False)**

Objetivo: Devuelve una copia ordenada de la lista de países.

Proceso interno: Utiliza sorted(). Si el campo es texto, ordena usando normalizar_texto() para evitar diferencias por mayúsculas. Si el campo es numérico, ordena directamente por el valor.

Interacción con otras funciones: Es llamada desde menu_ordenamientos() y no modifica la lista original.

Conceptos aplicados: sorted, lambda, parámetros opcionales, listas, ordenamientos.

- **mostrar_estadisticas(paises)**

Objetivo: Calcula y muestra indicadores generales del dataset.

Proceso interno: Primero verifica que existan datos. Luego calcula el país con mayor población usando max(), el de menor población con min(), promedios con sum() y len(), y cantidad por continente mediante calcular_cantidad_por_continente().

Interacción con otras funciones: Interactúa con mostrar_pais(), mostrar_linea(), formatear_numero() y calcular_cantidad_por_continente().

Conceptos aplicados: max, min, sum, len, acumuladores, diccionarios, estadísticas básicas.

- **main()**

Objetivo: Coordina la ejecución completa del programa.

Proceso interno: Carga los datos, muestra el menú principal y ejecuta la función correspondiente según la opción ingresada por el usuario. Se mantiene activo con while True hasta que el usuario elige salir.

Interacción con otras funciones: Es el centro de control del sistema. Desde main se conectan la interfaz, la lógica de negocio y la persistencia.

Conceptos aplicados: while, condicionales, llamadas a funciones, menú persistente, flujo de control.

9. Bibliografía

[1] Python Software Foundation. The Python Tutorial: Data Structures. Disponible en:
<https://docs.python.org/3/tutorial/datastructures.html>

[2] Python Software Foundation. csv - CSV File Reading and Writing. Disponible en:
<https://docs.python.org/3/library/csv.html>

[3] Python Software Foundation. Errors and Exceptions / Built-in Exceptions. Disponible en:
<https://docs.python.org/3/tutorial/errors.html> y
<https://docs.python.org/3/library/exceptions.html>

[4] Python Software Foundation. PEP 8 - Style Guide for Python Code. Disponible en:
<https://peps.python.org/pep-0008/>

Universidad Tecnológica Nacional. Carrera TUP 2026. *Guías de trabajos prácticos y documentación de la cátedra*. Material interno de la asignatura Programación I.

Universidad Tecnológica Nacional. Carrera TUP 2026. *Presentaciones, apuntes y material de estudio de Programación I*. Campus Virtual de la asignatura.